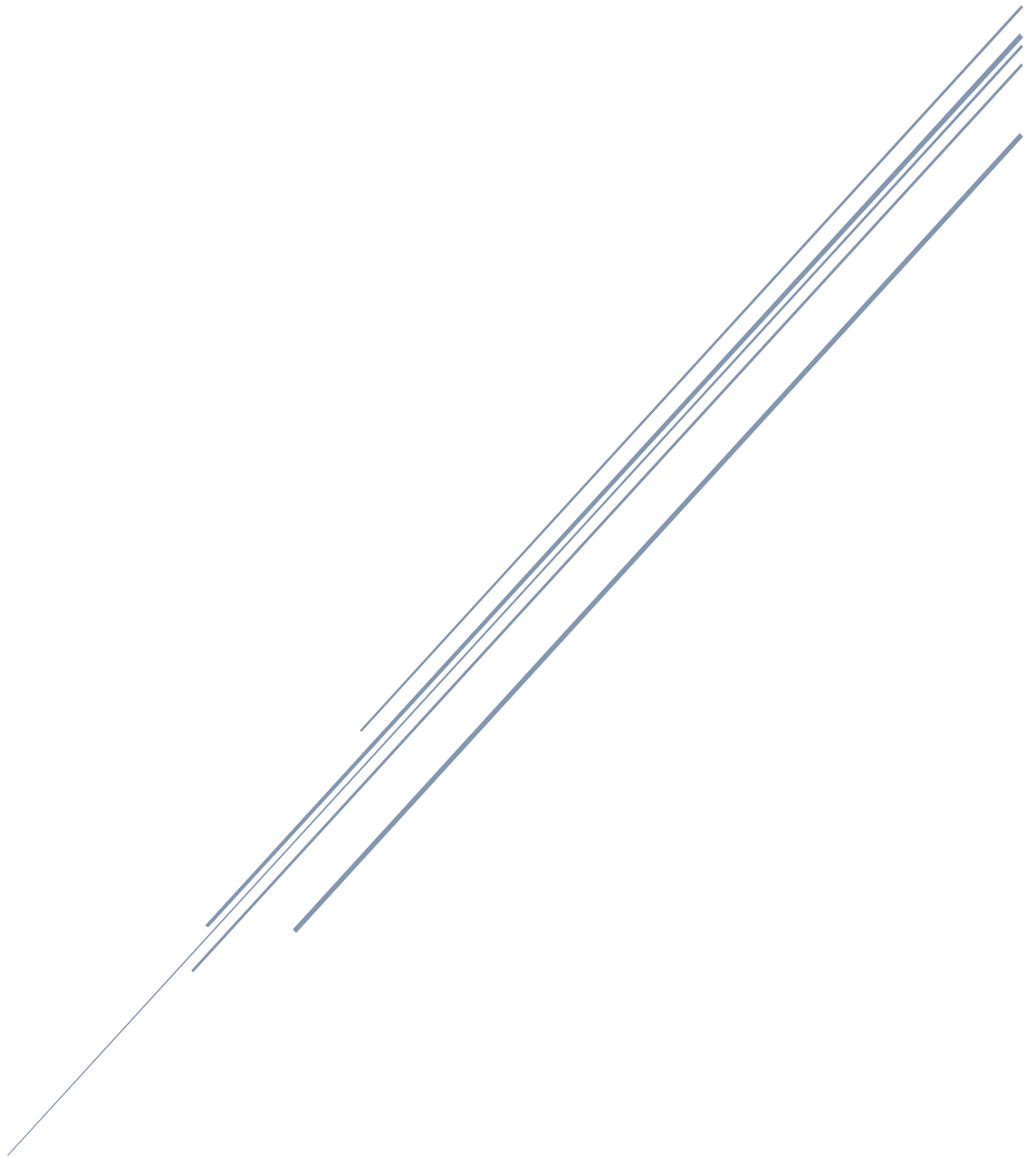


SPECIAL TOPICS IN AI

Deliverable 2: Retrieval Stack and NoSQL databases



Introduction

This project aims to build a comprehensive retrieval system and graph-informed system of information for scientific documents in PDF format. The first stage of development is to produce a large-scale process that can ingest research literature, pull content out of the documents, create a vector representation of the document, save associated metadata with each document, allow searching to be performed with low latency, and create a knowledge graph that reflects the relationships between research entities.

The developed solution integrates multiple technologies that provide a way to support both structured and unstructured information retrieval. Scientific documents in PDF format go through an ingestion pipeline to extract text and metadata, create document chunks that are manageable, generate embeddings, and save them to a series of databases; MongoDB is used for the storage of document and metadata, Qdrant provides the storage of vector embeddings and enables the ability to search using semantic retrieval, and Neo4j enables research papers/authors/topics/journal venue to be represented as relationships with one another.

A combined retrieval approach is achieved using both lexical and semantic search approaches producing an efficient hybrid retrieval mechanism. The combined hybrid approach creates a better quality of retrieval from original user input by providing a method for retrieving documents from an exact keyword match as well as allowing a user to gain a better understanding of the context of their query content. FastAPI provides a user-friendly interface to interact with the system, monitor the health of the database, assist in evaluating the retrieval performance for each document retrieved, and execute graph-based queries against the graph database to obtain information.

This project will demonstrate retrieving documents from a dataset containing more than (100) publically available scientific PDF documents covering a range of science/engineering research areas and providing a sufficient amount of variety to test retrieval accuracy, graph construction, and information discovery capabilities.

System Architecture and Implementation

The overall framework of the architecture includes four key elements: document ingestion, data storage, retrieval, and construction of a graph.

The document ingestion process starts with receiving PDF files that need to have their text extracted from them. Following this step, each PDF file is then split into overlapping chunks for contextually based resolution and lossy resolution. During this step, unique pieces of metadata are assigned to each individual chunk. This metadata holds information that provides access to citations for each academic document (title, source, identification number etc.) when performing a retrieval operation.

The final document and associated chunk record formats will be held in a database called MongoDB (the primary data store) where each single academic document will have one unique record within the MongoDB storage area. And, therefore, if there are multiple chunks associated with one academic document, these will be stored in a separate document on MongoDB. Each chunk record will also need to contain information about where the text originated from in order to increase the likelihood of efficient retrieval and citation generation.

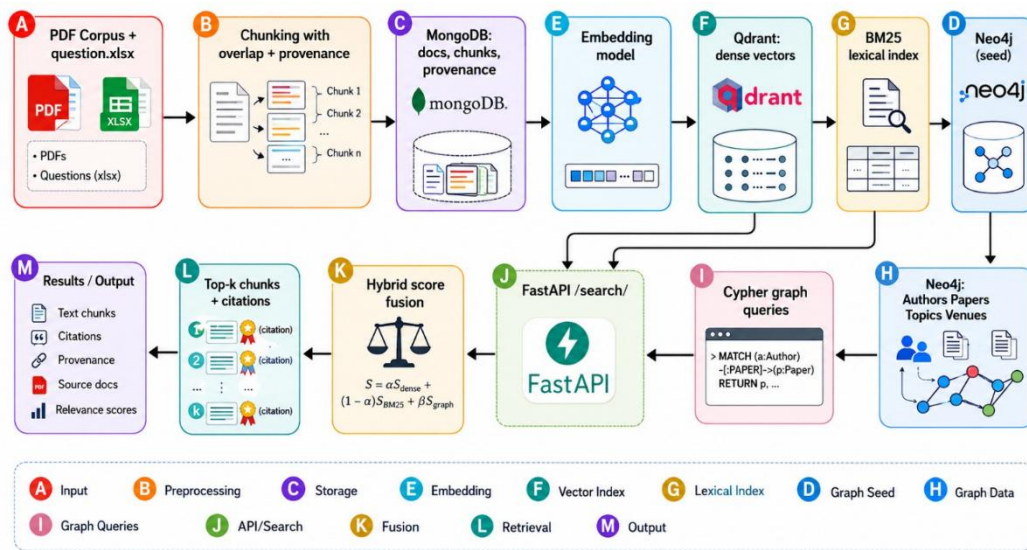
In support of semantic retrieval, the architecture has used a transformer-based embedding model to produce dense vector representations (embeddings) of chunk data. These embeddings will be stored in a vector-based similarity database called Qdrant (designed specifically for similarity searching). So, while the system will use embeddings to enable retrieval of chunk data when searching semantically. For instance, while the search terms do not match up exactly with any words in an academic document

the results of the queries will still enable user access to the same citations/source as the original documents.

The retrieval subsystem uses BM25 lexical methods to retrieve documents that contain keywords relevant to the subject being searched for and semantic methods to retrieve documents that mean the same as the query, then both are combined into one score with a weight on each method reflecting how important keyword relevance is versus how important semantic similarity is in producing a successful search outcome.

The graph database is developed with Neo4j and consists of four types of nodes that represent scientific papers, authors, topics and the venue where those papers are published, and edges that connect those nodes to show how they relate to each other. The edges allow users to navigate the graph and find the author of a certain paper, search for papers concerning a particular topic or look at trends in a publication venue over time. Graph traversal, relationship discovery and knowledge discovery are accomplished through the use of multiple Cypher queries showing examples of how the graph can be traversed.

The application layer of all system elements will function via the FastAPI API. The FastAPI API exposes endpoints for document ingestion and retrieval, supports graph queries, provides health checks on all system components, and exposes system statistics. The FastAPI API also has interactive documentation through the Swagger framework to facilitate developer testing and verification of all system features. All system elements - MongoDB, Qdrant, Neo4j, and FastAPI are deployed into Docker containers. Docker ensures reproducibility and simplifies the deployment of the overall system.



Retrieval Methodology

The retrieval system aims to improve the accuracy of searches as well as speed up the response time. A user query will use two different parallel retrieval methods to fulfill the request.

The first parallel method is BM25 lexical retrieval. BM25 uses keyword frequency and document statistics to compute the relevance of terms in a document collection. Therefore, if a user query has the exact words that are in the document collection, BM25 will usually perform very well.

The second parallel retrieval method is semantic retrieval. The user query is converted to an embedding vector with the same model that was used when ingesting the documents into Qdrant. Cosine similarity will be used to compare the embedding vector obtained during the user's information request to the embedding vectors of each document stored in the Qdrant system. Since embedding vectors represent only one aspect of word similarity, a user may use completely different words than documents stored in

Qdrant and still be able to retrieve similar concepts (document content) through Qdrant due to the use of word embeddings as a basis for determining how alike two documents are.

The outputs of both retrieval methods are normalized and merged via a weighted scoring method, resulting in a ranked list of chunks that are semantically and lexically relevant. Retrieved results will include metadata like titles of papers that contain the chunk, unique identifiers for the chunk itself (so it can be found again), page number references for where to find it in the original document, and similar citation information for users looking up citation information back to the original source.

Several different optimization techniques have been implemented to enhance performance. For example, the embedding models will be cached so there won't have to be repeated loading of embedding models for each time the search runs. Similarly, the MongoDB and Qdrant clients are cached to eliminate any overhead in connecting. BM25 indexes are saved from one retrieval session to the next and reused, thereby reducing processing time due to the fact that BM25 operations, such as indexing, are not required again for the same document. These optimizations have greatly reduced latency for retrieving documents while still providing retrieval quality.

Evaluation and Results

A set of 150 queries was used to carry out an evaluation of a retrieval system. The main goal of this evaluation was to assess the accuracy of the retrieved information and the speed of response. To do this, four metrics were used: Recall@1, Recall@3, Recall@5 and p95 latency.

Recall@1 indicates the proportion of these 150 queries that have a relevant document as the first result. For the remaining metrics, Recall@3 and Recall@5 measure the number of relevant documents returned in the first 3 and 5 results, respectfully. The above metrics provide a relative assessment of how well the retrieval system ranks relevant documents.

Metric	Value
Recall@1	0.7800
Recall@3	0.9467
Recall@5	0.9533
p95 Latency	0.5541 seconds
Number of Queries	150

Results indicate high performance on the evaluation dataset. The Recall@5 value of 0.9533 suggests that for nearly all (95%) of the evaluated queries, relevant information was retrieved as one of the top five results (with respect to retrieval performance). This indicates that our hybrid retrieval strategy effectively integrates both lexical and semantic retrieval capabilities.

Latency performance was also good, with an overall p95 latency of approximately 0.55 seconds, meaning that most of the search requests were completed in less than one second. These performance gains were achieved through caching, optimized parameters for search, and efficient access patterns to the database.

In addition, the evaluation process produced examples of retrieval that contained citations and source references. These examples demonstrate that not only does our system retrieve relevant results, but it also provides transparent provenance information that can be independently verified and validated by users.

Engineering and Deployment

With an eye toward future use and ease of usability, the goal of the project was to develop all components to be easily reproducible and maintainable. Docker Compose was utilized to provide orchestration of all systems' individual services, including MongoDB, Qdrant, Neo4j, and FastAPI. This means that any deployment of this project may be reproduced at any time, anywhere, precisely as it was originally deployed.

Throughout development, enhancements were added for performance and stability. Leveling the playing field between local or containerized deployments by separating Kubernetes environment variables and preventing connectivity issues. Caching of the database clients to eliminate initialization time and removal of duplicated files allowed for improved ingestion and storage time.

Within the repository, there is a systematic directory structure consisting of a programmatic logic folder (app), core functionality folder (src), utility scripts folder (scripts) and data files/ report folders. This structure increases both the readability of the overall project as well as provides ease for future maintenance and expansion.

The repository has all of the source files, configurations, evaluations, documentation and deployment necessary for complete reproduction of the system. The development process utilized GitHub as a method for version control, which provided an efficient way to track and share changes to the codebase during the development lifecycle.

Conclusion

The project above successfully implements an entire retrieval and graph-based information system for scientific PDF documents. The solution encompasses document ingestion, chunking, metadata storage into MongoDB, semantic vector storage into Qdrant, hybrid retrieval using BM25 and dense embeddings, Neo4j graph construction, FastAPI endpoints, Docker-based deployment, evaluation scripts, and citation-supported retrieval results.

The experimental evaluation indicates that the system is highly accurate and efficient and is yielding a retrieval performance of Recall@5 of 0.9533 and P95 latencies of 0.5541 seconds. The hybrid retrieval strategy effectively combines lexical and semantic search capabilities, while the graph database allows additional opportunities for knowledge discovery and relationship exploration.

The resulting architecture provides a scalable basis for future enhancements involving graph-guided retrieval, answer generation, higher reasoning capabilities, and broader scientific knowledge discovery applications.